AI Agents Trend and Opportunity Report

Executive summary

This report analyses the last five years of "AI agents" (roughly early 2021 through early 2026), then projects likely market and research trajectories through the next five years (roughly early 2026 through early 2031). It prioritizes practical, community-serving AI-agent services you could build, grounded in recent academic work, vendor documentation, public adoption metrics, and security/regulatory guidance. [1]

Key conclusions:

AI agents have moved from research prototypes to mass-market "agentic interfaces," but reliability and governance remain the bottleneck. Academic milestones such as tool-using reasoning patterns and self-taught tool use (for example, ReAct and Toolformer) made agents practical; meanwhile, benchmarks and systems papers exposed fragility in long-horizon and software tasks, pushing the field toward evaluation, tracing, guardrails, and durable execution. [2]

Adoption is real, but ROI uncertainty is also real. Enterprise survey data shows rapid growth in regular generative AI use in business functions, while analyst and press reporting highlights that many "agentic AI" projects fail to leave pilot stages due to cost, risk, and unclear value. [3]

High-use "agents" today cluster into three categories: consumer assistants at massive scale, developer/copilot agents at high daily frequency, and open-source "agent frameworks" (agent orchestration + tool integration) that have become infrastructure for the long tail of agentic apps. Public metrics indicate very large usage for ChatGPT-class assistants and GitHub Copilot-class developer tools, alongside extremely high adoption signals for open-source frameworks (stars, "used by," community activity). [4]

The most leverage for "help the AI community" services is in trust, evaluation, and interoperability. Security threats such as prompt injection are now treated as a first-class systems problem (not a "prompting" problem), and most teams still lack disciplined evaluation/regression testing for agent trajectories. Protocol standardization (notably MCP) is reducing fragmentation but creates new governance needs (tool registries, permissions, auditability, credential safety). [5]

Recommended MVP (highest combined impact, feasibility, and community benefit):

AgentSec Scanner: a security and compliance test harness for tool-using agents and MCP servers (open-source core + hosted CI). It addresses a widely cited top risk (prompt injection / tool misuse) and can be scoped to an MVP quickly: risk scoring, automated attack suites, sandbox permissions, and a "security report" artifact per agent/tool build. [6]

Scope and definitions

Working definition of an AI agent:

A software system that uses an AI model (typically a frontier LLM) to (a) interpret a goal, (b) plan or decompose tasks, (c) take actions via tools/APIs/UI automation, and (d) iterate with feedback (including memory, evaluation, or human input) until it reaches a stopping condition. This framing matches the trajectory from "reasoning + acting" research patterns into modern tool-calling and agent SDK documentation. [2]

Agent ecosystem layers used throughout this report:

Model/tool substrate: function calling, tool schemas, web/search tools, code execution, retrieval, and context protocols such as MCP. [7]

Agent frameworks: open-source libraries that orchestrate tool use, memory/state, multi-agent collaboration, tracing, durable workflows, and evaluation hooks. [8]

End-user agents: products people use daily (consumer assistants, coding agents, workflow agents), which often expose agentic behaviour such as "agent mode," background task execution, or agent flows. [4]

Primary sources prioritized in this report:

Analyst and survey sources: Gartner and McKinsey reports/surveys; Eurostat statistics. [9]

Vendor primary sources: Microsoft earnings materials; OpenAI docs and usage study; Anthropic protocol specs and pricing/system cards. [10]

Security and regulation: OWASP Top 10 for LLM apps; National Cyber Security Centre guidance; National Institute of Standards and Technology AI RMF; European Union AI Act text. [6]

Academic papers and benchmarks (recent): ReAct, Toolformer, AutoGen paper, SWE-bench, and OpenHands SDK paper. [2]

Five-year trend reality check

The last five years show a clear progression from "LLM as chatbot" toward "LLM as actor," and then toward "LLM as actor inside governed workflows."

Research milestones that made modern agents viable:

ReAct-style patterns (tight loops of reasoning + acting) and related approaches made it practical to interleave model reasoning with tool calls and environment feedback; this is an explicit shift away from "single-shot" prompting. [2]

Toolformer demonstrated that models can learn to use tools via self-supervision, which influenced the broader tooling direction (moving from hard-coded prompts toward learnable tool selection and standardized tool interfaces). [11]

Multi-agent frameworks and benchmarks rose because single-agent autonomy often fails on complex tasks. AutoGen formalized multi-agent conversation patterns and human-in-the-loop workflows, reflecting the field's move toward composition and orchestration rather than pure autonomy. [12]

Software-agent reality (coding agents) is now benchmarked and operationalized as an engineering discipline. SWE-bench provided an evaluation baseline, while subsequent discussions around benchmark leakage/compromise reinforced that agent evaluation must be treated like adversarial testing and continuous regression, not a one-time demo. [13]

Infrastructure milestones that shaped adoption:

Standardized tool/context connectivity is becoming a core "agent substrate." MCP was introduced explicitly to reduce fragmentation (custom integration per tool/model pairing) and to enable secure two-way connections between models and data/tools. [14]

Agent SDKs and tracing have become first-class expectations. OpenAI's agent documentation emphasizes agent handoffs, tools, guardrails, human-in-the-loop, sessions, and tracing as core concepts—reflecting the shift from "prompting" to "agent engineering." [7]

Security thinking matured: prompt injection sees as a systemic vulnerability ("inherently confusable deputy"), not something solved by better prompts alone, and OWASP continues to rank prompt injection among top LLM app risks. [5]

Market and research outlook through the next five years

This section combines hard forecasts (analyst statements, enterprise reporting) with clearly labelled inferences for 2030–2031, since public sources often forecast either "generative AI overall" or "agentic AI adoption in enterprise software," not an "agent market share" in the traditional sense. [15]

Near-term adoption signals and forecasts:

Gartner forecasts a large fraction of enterprise applications will embed conversational AI with proactive agents, indicating "agentic features" will become standard product surface area, not a niche app category. [9]

Reporting on Gartner research also highlights a key counterforce: large-scale pilot failure/cancellation driven by cost and unclear business value, and slower-than-hyped autonomy in real operations. [15]

Deloitte's outlook suggests broad corporate investment in "agentic AI" in the near term, which typically precedes more widespread production adoption if infrastructure and governance hurdles are resolved. [16]

Macroeconomic value potential sets the ceiling, not the timeline: McKinsey's estimates of generative AI value and automation potential indicate why organizations keep investing, but also why execution and change management dominate outcomes. [1]

Consumer-scale adoption also matters, because consumer assistants drive developer attention, talent formation, and an ecosystem of tools/plugins:

OpenAI's usage study describes a very large weekly active user base and analyses real conversation behaviour at scale, implying that "assistant/agent" usage is now a mainstream consumer behaviour (even if many interactions are still Q&A rather than autonomous task completion). [4]

Recent reporting indicates further growth in weekly active users into early 2026. [17]

Enterprise adoption context (important for 2026–2031 projections):

McKinsey survey reporting shows a sharp jump in regular generative AI usage in business functions from 2023 to 2024. This provides the "platform adoption curve" that agentic automation can ride on top of—agents are typically built after organizations already run genAI in production contexts. [3]

Eurostat reports substantial consumer exposure to generative AI tools in 2025, implying the general population's learning curve and comfort with AI assistance is developing rapidly—a tailwind for agent interfaces. [18]

Projections for the next five years (structured):

The table below mixes directly forecasted points (with citations) and scenario inferences (explicitly labelled) for early 2026 through early 2031.

Dimension	Likely direction	Forecast anchors	Practical implication for builders
Agentic features in enterprise software	Strong growth, becoming "default UI"	Gartner forecasts major embedding of proactive agent capabilities in enterprise applications by the near term. [9]	Demand shifts from "build an agent" to "operate agents safely at scale."
Agentic autonomy in production	Growth, but slower than hype	Gartner research (reported) projects meaningful penetration of agentic AI in enterprise software and some autonomy in daily	Services that reduce failure rates (eval, security, cost control) win.

		decisions by the late 2020s, while also warning that many projects are scrapped for cost/value reasons. [15]	
Corporate investment intensity	High in near term, then ROI-driven consolidation	Deloitte expects broad investment in agentic AI; McKinsey highlights large economic upside but emphasizes time and transformation requirements. [16]	SaaS buyers will demand measurable ROI and governance artifacts.
Standardization and interoperability	Rapid maturation	MCP was explicitly created to standardize tool/context integration and is being iterated in public specs. [14]	Tool registries, permissions, audit, and identity become first-class products.
Security risk surface	Rising, structurally hard	NCSC argues prompt injection is fundamentally different from classic injection; OWASP prioritizes prompt injection risk. [5]	"Agent security engineering" becomes a recurring budget line.
Regulation and compliance	Tightening, more explicit obligations	EU AI Act and NIST AI RMF provide governance frameworks and risk expectations that affect agent deployments. [19]	Compliance-ready logging, data controls, and human oversight become differentiators.
2030–2031 scenario inference	Agents are a normal layer in most knowledge workflows, but autonomy is still constrained by risk controls	Inference from the above adoption + governance trajectory (not a single cited forecast). [15]	The biggest opportunities are "agent operations," not "agent demos."

Adoption drivers and technical limitations

Adoption drivers that repeatedly show up across vendors, surveys, and engineering practice:

Distribution through existing platforms lowers friction. Microsoft's disclosures show both massive reach for developer tools and explicit strategic framing of agents as a new application model, which accelerates enterprise experimentation. [20]

Standardized connectivity reduces integration cost. MCP's explicit goal is to replace bespoke integrations with a universal "port" for context and tools; this changes the economics of building agent ecosystems and encourages third-party tooling. [14]

Rising baseline generative AI adoption provides "agent-ready" organizations. Surveys show rapid growth in regular genAI use, making it more likely that teams then attempt multi-step workflows, tool use, and automation. [3]

The economic motivation is enormous. McKinsey estimates large value potential and significant shares of work time composed of activities that generative AI could automate or accelerate, which creates persistent executive pressure to move beyond chat into workflow automation. [1]

Technical limitations that frequently prevent production success:

Long-horizon reliability and evaluation gaps. Benchmarks such as SWE-bench exist precisely because subjective demos are unreliable; subsequent discussions about benchmark leakage illustrate that evaluation itself must be hardened to remain meaningful. [13]

Prompt injection and tool hijacking are a core agent failure mode. NCSC frames prompt injection as an "inherently confusable deputy" issue; OWASP continues to highlight prompt injection as a top LLM application risk. This matters more for agents than for chatbots because agents have permissions and act on tools. [5]

Cost and unclear value kill pilots. Gartner research (reported) warns of high rates of project cancellation/repurposing due to cost and unclear business value—especially relevant for agents, which often require multiple model calls, tools, and monitoring. [15]

Over-eagerness and unsafe workaround behaviour in autonomous settings remains a concern even for leading models, motivating safeguard layers, monitoring, and human oversight. [21]

Regulatory and ethical risks (practically relevant for agent services):

System-level accountability and governance requirements are increasingly formalized. NIST AI RMF provides a risk management framing; EU AI Act creates a legal governance environment that affects high-risk use cases, transparency, and lifecycle obligations. [22]

Privacy and data handling intensity grows with agent capabilities. Agents routinely require access to enterprise data sources and tools (email, docs, tickets, code repos), which elevates the need for least privilege, auditability, data minimization, and clear retention boundaries. [14]

Landscape of widely used AI agents

The "AI agent" label is used for both agentic products (end-user) and agentic frameworks (developer infrastructure). To keep the catalogue actionable, this section separates the

two and prioritizes “widely used” by public usage signals (users, seats, GitHub stars, “used by” counts, or equivalent). [14]

Open-source agent frameworks and orchestration libraries (developer infrastructure):

Framework	Core capabilities	Deployment contexts	Typical personas	Usage metrics (public)
LangChain	Integrations + agent abstractions for LLM apps; broad ecosystem	Local dev → cloud services	App engineers, prototypers, startups	~128k GitHub stars; ~277k “used by” repos (GitHub). [8]
LangGraph	Graph-based, stateful, durable agent workflows; human-in-loop patterns	Long-running agents in production	Agent engineers, platform teams	~25.6k stars; ~36.5k “used by.” [23]
LlamaIndex	“Agents over data” toolkit + broad integration packs	RAG + document agents; enterprise ingestion	Data/ML engineers, RAG builders	~47.4k stars (GitHub). [24]
AutoGen	Multi-agent applications; human-in-loop; agent teaming patterns	Research + production prototypes	Researchers, agent system builders	~55.2k stars; ~4k “used by.” [25]
CrewAI	Multi-agent orchestration (“crews” + “flows”); enterprise automation posture	Workflow automation; enterprise control planes	Automation engineers, consultants	~45.1k stars; ~17.9k “used by.” [26]
Semantic Kernel	Model-agnostic agent SDK; plugins; multi-agent orchestration	Enterprise apps; .NET/Python/Java	Enterprise devs, platform teams	~27.4k stars; ~2.5k “used by.” [27]
Haystack	Modular pipelines + agent workflows, explicit retrieval/memory control	Production RAG/agent pipelines	ML engineers, search/RAG teams	~24.4k stars; ~1.3k “used by.” [28]
OpenAI Agents SDK	Provider-agnostic multi-agent workflows, guardrails, tracing	Tool-using agents with tracing	Agent builders, product teams	~19.3k stars (GitHub). [29]

Open-source agentic products and "autonomous-ish" tools (end-user / dev-user):

Agent project	What it does well	Deployment contexts	Personas	Usage metrics (public)
AutoGPT	Early autonomous agent platform + ecosystem; agent protocol alignment	Self-hosted; experimentation	Hobbyists, prototypers	~182k stars; ~46.2k forks. [30]
OpenHands	Software development agents (CLI/GUI/cloud); agent SDK	Local + hosted; code execution	Engineers, OSS maintainers	~68.6k stars; active releases. [31]
Open Interpreter	Local code execution + computer control with approvals	Local machine	Power users, analysts	~62.5k stars. [32]
Aider	Terminal-based pair programming with VCS integration	Local dev workflows	CLI-first developers	~41.4k stars. [33]
Continue	Source-controlled "agent checks" in CI + IDE tooling	CI pipelines + IDE	Dev teams, QA/security	~31.6k stars. [34]

Commercial, high-use agents (end-user products with public metrics):

This list focuses on products with disclosed scale signals. Many commercial agents do not publish global MAU/WAU.

ChatGPT-scale consumer assistants. OpenAI's usage study references a very large weekly active user base as of a late-2025 study; additional reporting indicates further growth into early 2026. [4]

Developer copilots. Microsoft's annual report states GitHub Copilot has more than [20] million users; Microsoft's FY26 Q2 earnings materials report 4.7 million paid Copilot subscribers (across Copilot offerings) and 15 million paid Microsoft 365 Copilot seats, demonstrating massive distribution and monetization efforts. [20]

Standardization push. MCP is positioned as an open standard for secure tool/data connectivity across AI-powered tools; adoption of this layer affects both open-source and commercial agent ecosystems. [14]

Unmet needs and viable AI-agent services to build

The agent ecosystem's unmet needs are not "more agents" in the abstract—they are the missing safety, evaluation, and interoperability primitives that allow agents to be deployed responsibly and economically.

Unmet needs (evidence-based):

Secure-by-design tool use remains unsolved. Prompt injection is fundamentally hard to eliminate and needs impact-reduction architectures; OWASP and NCSC both emphasize this as a top priority. [5]

Evaluation is under-institutionalized. Benchmarks exist, but real product teams still struggle with regression testing of agent trajectories, tool-use reliability, and benchmark contamination. [13]

Interoperability is improving but creates governance gaps. MCP reduces bespoke integration cost, but it also increases the need for registries, permissions, identity, auditing, and security controls around third-party servers and tool schemas. [14]

Cost/ROI management is a make-or-break adoption driver. Gartner research (reported) indicates many agentic AI projects are scrapped due to cost and unclear value, making cost observability and value measurement a direct market need. [15]

Prioritization matrix for the proposed services:

Service ideas (each includes problem, users, requirements, effort, monetization, community value, risks):

AgentSec Scanner (recommended MVP)

Problem: teams ship tool-using agents without systematic testing for prompt injection, tool hijacking, over-privilege, data exfiltration, or unsafe side effects.

Target users: agent framework maintainers, agent app teams, MCP server developers, enterprise security teams.

Technical requirements: attack suite runner; tool permission sandbox; taint-like tracking for “untrusted text”; policy engine; static checks for tool schemas; generation of signed security reports; optional CI integration.

Data needs: curated prompt-injection and tool-injection corpora; synthetic adversarial tasks; logs of tool calls for replay. [35]

Estimated development effort: MVP in ~6–10 weeks for a tight scope (MCP server scanning + prompt-injection suite + report generator); production SaaS in ~4–6 months.

Monetization options: hosted CI pricing per scan/run; enterprise compliance package; open-core with paid “policy packs” and integrations.

Community value: raises baseline security hygiene; creates comparable security artifacts across frameworks; reduces catastrophic agent misuse.

Risks: false confidence from incomplete tests; handling sensitive logs; adversarial misuse of the scanner itself. Mitigate via clear "coverage limits," secure log handling, and safe red-teaming practices. [5]

EvalOps CI for agent regression

Problem: agent behaviour regresses silently when prompts/models/tools change; teams lack unit tests for multi-step trajectories.

Target users: OSS maintainers, startups shipping agentic features, research labs.

Technical requirements: scenario DSL; tool mocking/simulation; deterministic replay; evaluation metrics (task success, tool-call correctness, safety constraints); reporting and trend dashboards.

Data needs: benchmark tasks (web, code, business workflows) + golden traces; difficult because benchmarks can be gamed, so continuous refresh is needed. [13]

Effort: MVP ~8–12 weeks; credible benchmark library ~6–12 months (ongoing).

Monetization: hosted test runners; enterprise private benchmark hosting; consulting for evaluation design.

Community value: makes "agent engineering" more like software engineering.

Risks: benchmark leakage; overfitting; high infra costs for frequent test runs. [36]

MCP Tool Registry with trust, permissions, and governance

Problem: MCP lowers integration friction, but the ecosystem needs a trust layer: server discovery, permission scopes, identity, audit signals, and security posture.

Target users: MCP server builders, agent app builders, enterprise platform teams.

Technical requirements: registry service; verification pipeline; schema linting; OAuth/identity support; risk scoring; signed metadata; optional "allowlist" distribution.

Data needs: server metadata, security attestations, incident reports, version history. [14]

Effort: MVP ~10–14 weeks (basic registry + verification + signing); scaled ecosystem effort is longer.

Monetization: paid verification tiers; enterprise registry mirror; marketplace revenue share.

Community value: reduces fragmentation and unsafe integrations; accelerates adoption.

Risks: becoming a central point of failure; liability for "verified" claims.

Cross-framework observability and cost control

Problem: teams can't manage what they can't see—agent runs are opaque, expensive, and hard to debug across frameworks/providers.

Target users: agent platform teams, startups, researchers.

Technical requirements: OpenTelemetry-compatible traces; adapters for major frameworks; token/tool cost accounting; "run diffing" for regression; privacy-preserving logging.

Data needs: trace/event schema; sampled run logs; cost metadata from providers. [29]

Effort: MVP ~8–12 weeks for a single framework + provider set; broader coverage is ongoing.

Monetization: hosted dashboards; usage-based pricing; enterprise on-prem.

Community value: reduces wasted spend; speeds up debugging.

Risks: sensitive data in traces; vendor lock-in concerns.

AgentSim environments for robust evaluation

Problem: agents fail in realistic, interactive environments; current evaluation tasks are too synthetic or too narrow.

Target users: academic labs, agent builders, safety teams.

Technical requirements: controllable simulated environments (web UI tasks, ticketing, code repo, data entry); scenario generator; scoring.

Data needs: task templates; realistic UI state; failure cases; adversarial scenarios. [13]

Effort: MVP ~12–16 weeks for a narrow domain; expansion is substantial.

Monetization: hosted simulation runs; dataset licensing; enterprise scenario packs.

Community value: bridges the gap between benchmarks and production tasks.

Risks: environment maintenance; benchmark gaming.

User-owned memory vault

Problem: agent "memory" today is often stuck inside proprietary apps or logs, creating privacy risk and lock-in.

Target users: privacy-conscious users, developers building personal agents, enterprises with strict data controls.

Technical requirements: encrypted memory store; retrieval constraints; consent/retention controls; per-tool data scoping; audit logs.

Data needs: user data (sensitive); requires strong privacy/security posture. [19]

Effort: MVP ~10–14 weeks for encrypted store + API; enterprise-grade is larger.

Monetization: paid hosted vault; enterprise licensing; open-core client libraries.

Community value: healthier agent ecosystem with user control.

Risks: security breach; regulatory exposure.

Verifier-as-a-service for tool outputs

Problem: tool-using agents hallucinate, mis-handle edge cases, or take unsafe shortcuts; teams need "second-pass" validators.

Target users: agent app teams, safety teams.

Technical requirements: validator pipeline; deterministic checks (schema validation, reconciliation, unit tests); model-based critique; safety policy enforcement.

Data needs: tool schemas; expected invariants; test corpora. [21]

Effort: MVP ~6–10 weeks.

Monetization: per-validation call; enterprise policy packs.

Community value: reduces real-world failures, improves trust.

Risks: validators become another opaque model layer; latency/cost overhead.

Synthetic tool-use dataset pipeline

Problem: training and evaluation data for tool use and long-horizon tasks is scarce and quickly becomes stale.

Target users: open-source model builders, academic labs, agent framework maintainers.

Technical requirements: data generation pipeline with sandboxed tools; annotation; quality filters; release process.

Data needs: sandbox environments; tool sets; careful licensing. [11]

Effort: MVP ~10–14 weeks; ongoing curation long-term.

Monetization: hosted generation; dataset sponsorships; enterprise private datasets.

Community value: accelerates open research and reproducible benchmarks.

Risks: misuse; licensing errors; benchmark gaming.

Prioritized ranking (impact, feasibility, community benefit scored on a 1–10 scale; these are analytic estimates, not cited "facts"):

Rank	Service idea	Impact	Feasibility	Community benefit	Why it ranks here
1	AgentSec Scanner	9	7	9	Directly addresses top systemic risk (prompt/tool injection) with a tractable MVP; strong adoption pull from security guidance. [5]
2	EvalOps CI	8	8	9	Converts agent building into disciplined engineering; ties to benchmark lessons and enterprise "pilot failure" reality. [36]
3	Verifier-as-a-service	8	7	8	Improves reliability with modular checks; maps to safety concerns in autonomous settings. [21]
4	Cross-framework observability	7	7	8	High leverage for cost + debugging; aligns with agent SDK emphasis on tracing. [29]
5	MCP Tool Registry + trust	7	6	8	Solves fragmentation, but governance + network effects are harder. [14]
6	AgentSim environments	8	6	8	Valuable but heavier engineering + maintenance. [37]
7	User-owned memory vault	6	6	7	Important but security + trust challenges; slower adoption. [19]
8	Synthetic tool-use dataset pipeline	6	5	7	Useful but needs sustained curation to stay relevant. [36]

Cost comparison table (example unit economics for agent services):

Below is a practical "back-of-the-envelope" cost model—useful for MVP planning—based on publicly posted token pricing and tool costs. Actual costs depend heavily on prompt length, tool loops, retries, and caching. [38]

Assume a single agent task averages 40k input tokens and 8k output tokens (multi-step tool use), and you run 10,000 tasks/month.

Provider example	Input price (per MTok)	Output price (per MTok)	Estimated model cost / task	Estimated model cost / 10k tasks
OpenAI GPT-5.2	$1.75 USD	$14.00 USD	(0.04×1.75) + (0.008×14) ≈ $0.18 USD	≈ $1,800

Anthropic Sonnet 4.6	$3.00 USD	$15.00 USD	(0.04×3) + (0.008×15) ≈ $0.24 USD	≈ $2,400
Anthropic Opus 4.6	$5.00 USD	$25.00 USD	(0.04×5) + (0.008×25) ≈ $0.40 USD	≈ $4,000

Notes:

Cloud tool costs can dominate in some workloads. For example, OpenAI File Search pricing includes vector storage per GB per day and per-query costs, which matters if your service evaluates agents against large corpora. [39]

Prompt caching and batch processing can materially reduce costs on some platforms, affecting SaaS margins if you design for reuse. [40]

MVP plans and architectures

This section details the top three MVPs (based on the prioritization) with timelines and reference architectures. It also proposes business models aligned with community value (open-source + hosted options) and the dominant risk profile (security, privacy, and evaluation integrity).

Recommended MVP: **AgentSec Scanner**

MVP scope (tight and shippable):

MCP server scanner: validate tool schemas, permissions patterns, OAuth configuration, and “risky defaults.”

Prompt injection test suite: replay adversarial prompts against an agent + tool harness, measure leakage, unauthorized tool calls, privilege escalation attempts, and unsafe output.

Artifact output: signed “Agent Security Report” (JSON + human-readable) that can be published in CI or attached to releases.

Why this MVP is highest leverage now:

Security risk is systemic and widely recognized (NCSC and OWASP), and it grows with adoption of tool-connected agents and MCP ecosystems. [5]

Enterprises are already auditing massive volumes of Copilot interactions, indicating strong demand for governance artifacts and auditable controls. [10]

Second MVP: **EvalOps CI for agent regression**

MVP scope:

Scenario definitions for a narrow domain (for example: “tool calling + search + summarization” or “PR code review agent”)

Deterministic replay harness with mock tools and golden traces

Metrics: task success, tool-call correctness, policy violations, cost per success

Third MVP: **Verifier-as-a-service**

MVP scope:

Schema + invariant checks for tool outputs (JSON schema validation, reconciliation rules)

Optional “second model critic” with strict budget and no tool permissions

Policy enforcement hook compatible with common agent SDKs/frameworks

Business models aligned to community value

Open-core is the default-fit model for community-serving infrastructure: open-source cores build trust and adoption; hosted services monetize convenience, scaling, enterprise features (SAML, audit exports), and continuous updates. This is consistent with how agent platforms position their own “enterprise control plane” offerings. [26]

Usage-based pricing can be aligned to real costs (tokens, tool calls). Vendors increasingly disclose token economics and cost management advice, indicating buyers will compare cost per outcome. [41]

Enterprise licensing (on-prem / VPC) is often required for regulated data and aligns with the governance expectations emphasized by security and risk frameworks. [22]

Risk register for the recommended MVP (AgentSec Scanner)

Technical risk: incomplete coverage leading to a false sense of security. Mitigation: explicit coverage labels; continuous updates; encourage defence-in-depth patterns rather than “pass/fail.” [5]

Operational risk: handling sensitive prompts/logs. Mitigation: local-first execution option; encryption; partial redaction; enterprise on-prem modes; retention controls aligned to risk frameworks. [22]

Adversarial risk: attackers use the tool to refine attacks. Mitigation: rate limiting in hosted version; safe disclosure; differential access for advanced packs; avoid publishing exploit details beyond responsible guidance. [6]

Citations

mckinsey.com

1 https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-economic-potential-of-generative-ai-the-next-productivity-frontier

3 https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai-how-organizations-are-rewiring-to-capture-value

arxiv.org

2 https://arxiv.org/abs/2210.03629https://arxiv.org/abs/2210.03629

11 https://arxiv.org/abs/2308.08155https://arxiv.org/abs/2308.08155

12 https://arxiv.org/abs/2302.04761https://arxiv.org/abs/2302.04761

13 https://arxiv.org/abs/2410.06992https://arxiv.org/abs/2410.06992

35 https://arxiv.org/abs/2302.12173https://arxiv.org/abs/2302.12173

37 https://arxiv.org/html/2511.03690v1https://arxiv.org/html/2511.03690v1

openai.com

4 https://openai.com/index/how-people-are-using-chatgpt/

38 https://openai.com/api/pricing/https://openai.com/api/pricing/

39 https://openai.com/index/new-tools-and-features-in-the-responses-api/

ncsc.gov.uk

5 https://www.ncsc.gov.uk/blog-post/prompt-injection-is-not-sql-injection

genai.owasp.org

6 https://genai.owasp.org/llmrisk/llm01-prompt-injection/

developers.openai.com

7 https://developers.openai.com/api/docs/assistants/tools/

github.com

8 https://github.com/langchain-ai/langchain

23 https://github.com/langchain-ai/langgraph

24 https://github.com/run-llama/llama_index

25 https://github.com/microsoft/autogen

26 https://github.com/crewAIInc/crewAI

27 https://github.com/microsoft/semantic-kernel

28 https://github.com/deepset-ai/haystack

29 https://github.com/openai/openai-agents-python

30 https://github.com/Significant-Gravitas/AutoGPT

31 https://github.com/OpenHands/OpenHands

32 https://github.com/openinterpreter/open-interpreter

33 https://github.com/Aider-AI/aider

34 https://github.com/continuedev/continue

gartner.com

9 https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-from-less-than-5-percent-in-2025

microsoft.com

10 https://www.microsoft.com/en-us/investor/events/fy-2026/earnings-fy-2026-q2

20 https://www.microsoft.com/investor/reports/ar25/index.html

anthropic.com

14 https://www.anthropic.com/news/model-context-protocol

21 https://anthropic.com/claude-sonnet-4-6-system-card

reuters.com

15 https://www.reuters.com/business/over-40-agentic-ai-projects-will-be-scrapped-by-2027-gartner-says-2025-06-25/

deloitte.com

16 https://www.deloitte.com/us/en/about/press-room/deloitte-2026-tmt-predictions.html

techcrunch.com

17 https://techcrunch.com/2026/02/27/chatgpt-reaches-900m-weekly-active-users/

insentragroup.com

18 https://www.insentragroup.com/nz/insights/not-geek-speak/generative-ai/agentic-ai-takes-the-wheel-a-deep-dive-into-2026/

eur-lex.europa.eu

19 https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng

nvlpubs.nist.gov

22 https://nvlpubs.nist.gov/nistpubs/ai/nist.ai.100-1.pdf

dl.acm.org

36 https://dl.acm.org/doi/10.1145/3605764.3623985

docs.anthropic.com

40 https://docs.anthropic.com/de/docs/about-claude/pricing

41 https://docs.anthropic.com/en/docs/claude-code/costs